# CS 255 Model Application Short Paper

Tim Vue

tim.vue@snhu.edu

Southern New Hampshire University

**Process Model Application**

I would apply a process-modeling approach to DriverPass by mapping how each service begins, what information moves through the system, who performs each action, and how the process ends. The first model should show customer registration from a phone call, office visit, or website entry through validation, account creation, package selection, and payment. Other models should describe appointment management, online course access, practice tests, instructor notes, password recovery, reports, and DMV updates. This would make the workflow visible before the team organizes the software.

Scheduling is especially important because it combines several business rules. A customer or secretary selects a date and time for a two-hour lesson. The system checks the student's remaining package hours and verifies that the customer, a driver, and a vehicle are available. It records the matching pickup and drop-off location, creates the reservation, and identifies its creator. Modifications and cancellations must preserve the audit history. Alternate paths should cover conflicts, missing information, unavailable resources, payment problems, and insufficient package hours.

Process models would also describe training and administrative information. A practice-test process would move a customer from selecting a test through answering questions, calculating the score, and saving the correct status. A DMV-update process would receive new material, notify authorized staff, support review, and publish approved content. Reporting would show how the owner selects activity data and displays, prints, or exports the result. These models let DriverPass confirm that the sequence matches daily work and addresses important exceptions.

## Object Model Application

I would apply object modeling by identifying the people, resources, transactions, and training records DriverPass must represent. A general User class could contain an identifier, contact information, credentials, account status, and authentication behavior. Customer, Secretary, Instructor, Owner, and IT Administrator classes could inherit shared features while adding role-specific attributes or operations. This supports different permissions without duplicating login and contact fields.

Central objects would include TrainingPackage, Enrollment, Appointment, Vehicle, Course, PracticeTest, TestAttempt, LessonNote, Payment, Notification, DMVUpdate, and AuditEntry. Each appointment would reference one customer, instructor, and vehicle and store its time, locations, status, creator, and modifier. A package would define included driving hours, classroom instruction, and online access, while an enrollment would preserve one customer's selection and remaining services. This lets DriverPass disable a package for new sales without changing prior purchases.

Object behavior would enforce rules close to the data it protects. Appointment operations could checkAvailability(), schedule(), reschedule(), and cancel(), while TestAttempt could start(), submit(), calculateScore(), and updateStatus(). Administrators could disable accounts, reset passwords, and assign roles. Payment records should use secure tokens and transaction references rather than readable card numbers or security codes. Associations would show that a vehicle or instructor supports many appointments over time but only one during a particular time period.

**Process and Object Model Comparison**

A process model explains DriverPass from the viewpoint of work being completed. Business users can follow its triggers, decisions, actions, handoffs, and outcomes. It suits reservation conflicts, password resets, DMV notifications, and approvals because alternate and failure paths are visible. It also helps the team find missing steps and parallel work. Its disadvantage is limited data detail. It may show an appointment being created without defining every field, relationship, or reusable software behavior.

The object model gives developers a stable view of classes, attributes, methods, inheritance, and relationships. It clarifies how customers, employees, packages, appointments, resources, tests, and audit records connect. That structure supports database design and reduces duplication through shared parent classes. Its disadvantage is a limited view of time and sequence. A class diagram can relate an Appointment to a Customer and Vehicle without showing how the system finds an open time, handles a conflict, notifies the customer, or preserves a cancellation trail.

DriverPass should therefore use both approaches. The process model should establish how users complete each business task and should expose decisions, dependencies, and exception paths. The object model should translate the important nouns and rules from those workflows into maintainable software components and database entities. The team can validate the process models with business users, then check that every process has the objects, attributes, and operations needed to support it. The reverse check is also useful: every class and operation should support a real requirement or workflow. Used together, the two models provide a more complete design than either model could provide alone.